

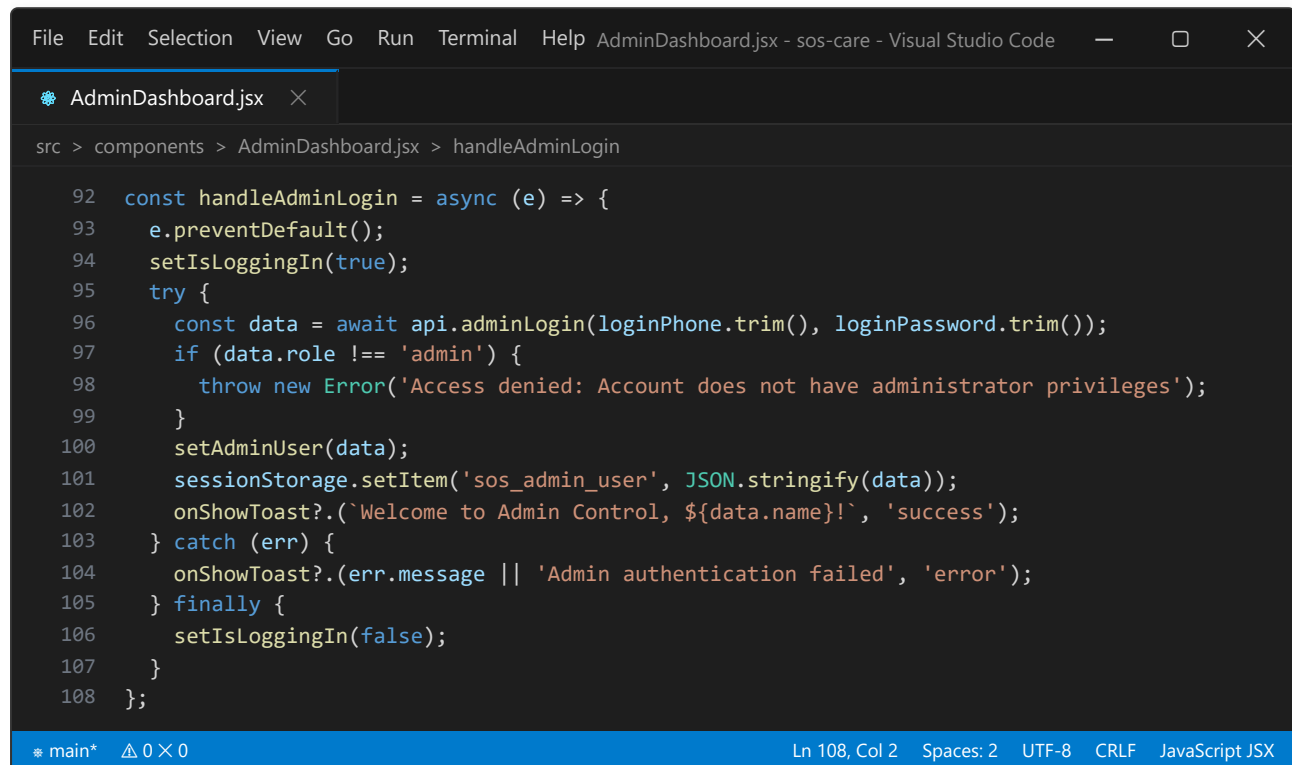
Mohd Salman Akther Khan Sabit

2321132642

Week 6:

1. Isolated Administrator Gateway & Session Management (AdminDashboard.jsx):

- Built an isolated administrator control portal with standalone login authentication (sos_admin_user), role validation, and multi-portal tab navigation.



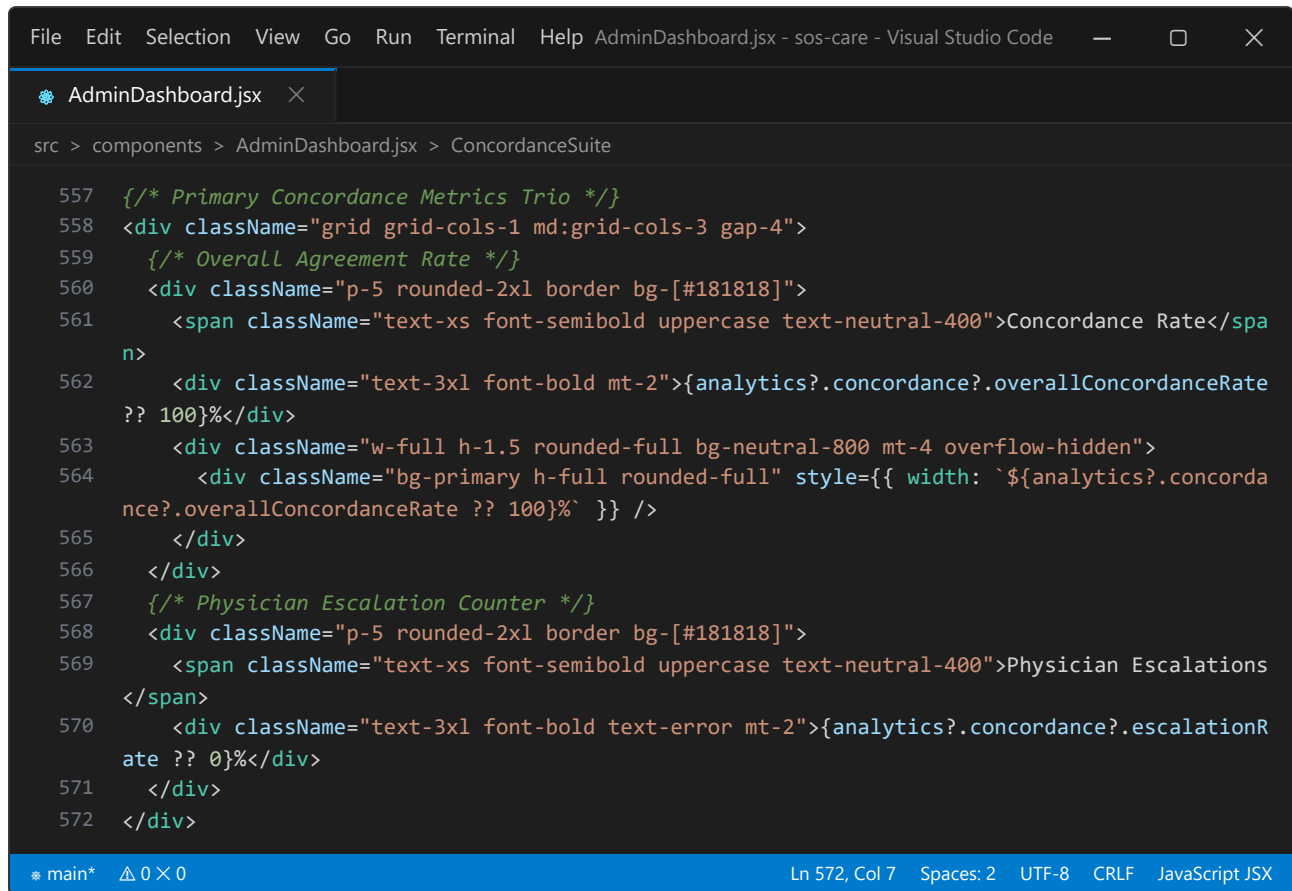
```
File Edit Selection View Go Run Terminal Help AdminDashboard.jsx - sos-care - Visual Studio Code
AdminDashboard.jsx
src > components > AdminDashboard.jsx > handleAdminLogin

  92  const handleAdminLogin = async (e) => {
  93    e.preventDefault();
  94    setIsLoggingIn(true);
  95    try {
  96      const data = await api.adminLogin(loginPhone.trim(), loginPassword.trim());
  97      if (data.role !== 'admin') {
  98        throw new Error('Access denied: Account does not have administrator privileges');
  99      }
 100      setAdminUser(data);
 101      sessionStorage.setItem('sos_admin_user', JSON.stringify(data));
 102      onShowToast?.(`Welcome to Admin Control, ${data.name}!`, 'success');
 103    } catch (err) {
 104      onShowToast?.(err.message || 'Admin authentication failed', 'error');
 105    } finally {
 106      setIsLoggingIn(false);
 107    }
 108  };

* main* 0x0 Ln 108, Col 2 Spaces: 2 UTF-8 CRLF JavaScript JSX
```

2. Clinical Triage Concordance & Diagnostic Agreement Suite (AdminDashboard.jsx):

- Developed clinical concordance analytics calculating diagnostic agreement rates (%) between automated triage evaluations and attending physicians, including escalation rates and discrepancy tables.



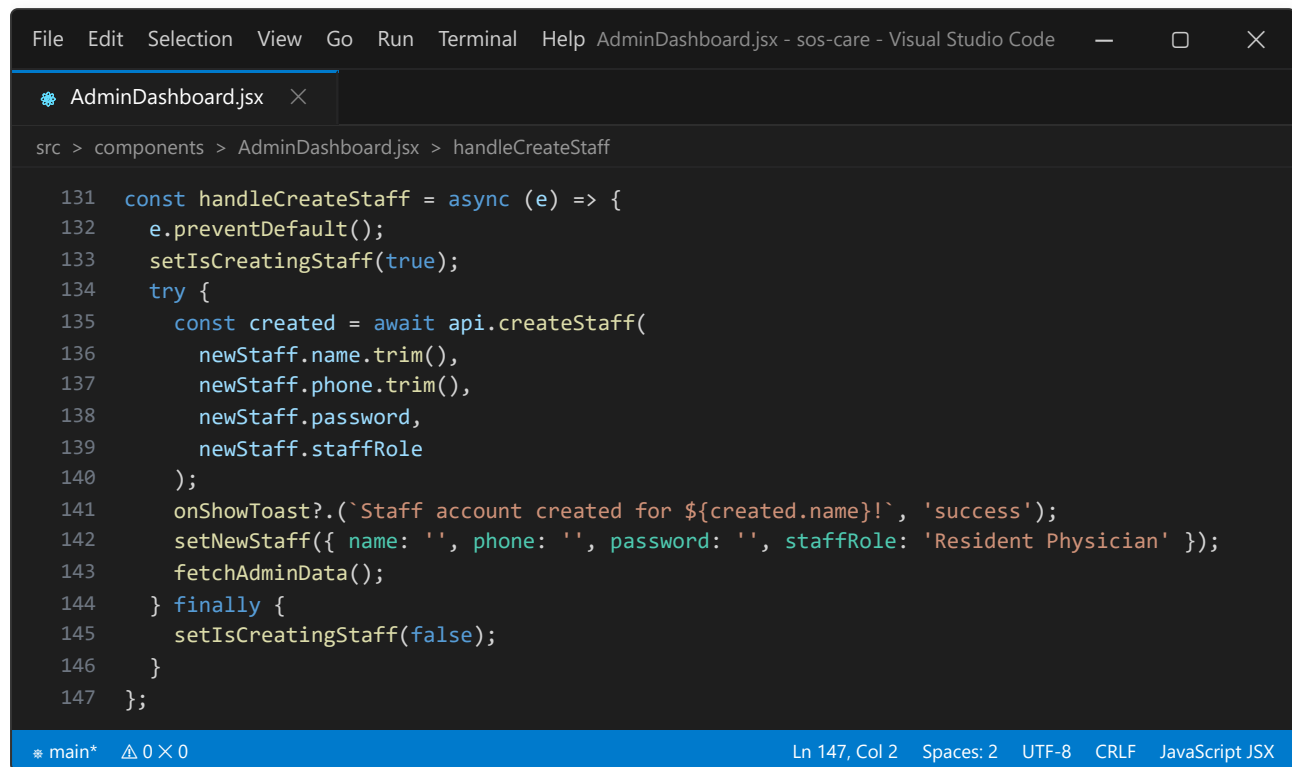
```
File Edit Selection View Go Run Terminal Help AdminDashboard.jsx - sos-care - Visual Studio Code
AdminDashboard.jsx
src > components > AdminDashboard.jsx > ConcordanceSuite

557  /* Primary Concordance Metrics Trio */
558  <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
559    /* Overall Agreement Rate */
560    <div className="p-5 rounded-2xl border bg-[#181818]">
561      <span className="text-xs font-semibold uppercase text-neutral-400">Concordance Rate</span>
562      <div className="text-3xl font-bold mt-2">{analytics?.concordance?.overallConcordanceRate ?? 100}%</div>
563      <div className="w-full h-1.5 rounded-full bg-neutral-800 mt-4 overflow-hidden">
564        <div className="bg-primary h-full rounded-full" style={{ width: `${analytics?.concordance?.overallConcordanceRate ?? 100}%` }} />
565      </div>
566    </div>
567    /* Physician Escalation Counter */
568    <div className="p-5 rounded-2xl border bg-[#181818]">
569      <span className="text-xs font-semibold uppercase text-neutral-400">Physician Escalations</span>
570      <div className="text-3xl font-bold text-error mt-2">{analytics?.concordance?.escalationRate ?? 0}%</div>
571    </div>
572  </div>

* main* 0 x 0 Ln 572, Col 7 Spaces: 2 UTF-8 CRLF JavaScript JSX
```

3. Staff Account Provisioning & User Directory (AdminDashboard.jsx):

- Implemented a staff provisioning module supporting hospital designations (Chief Nephrologist, Resident Physician, Triage Nurse) alongside a searchable, filterable user management directory.



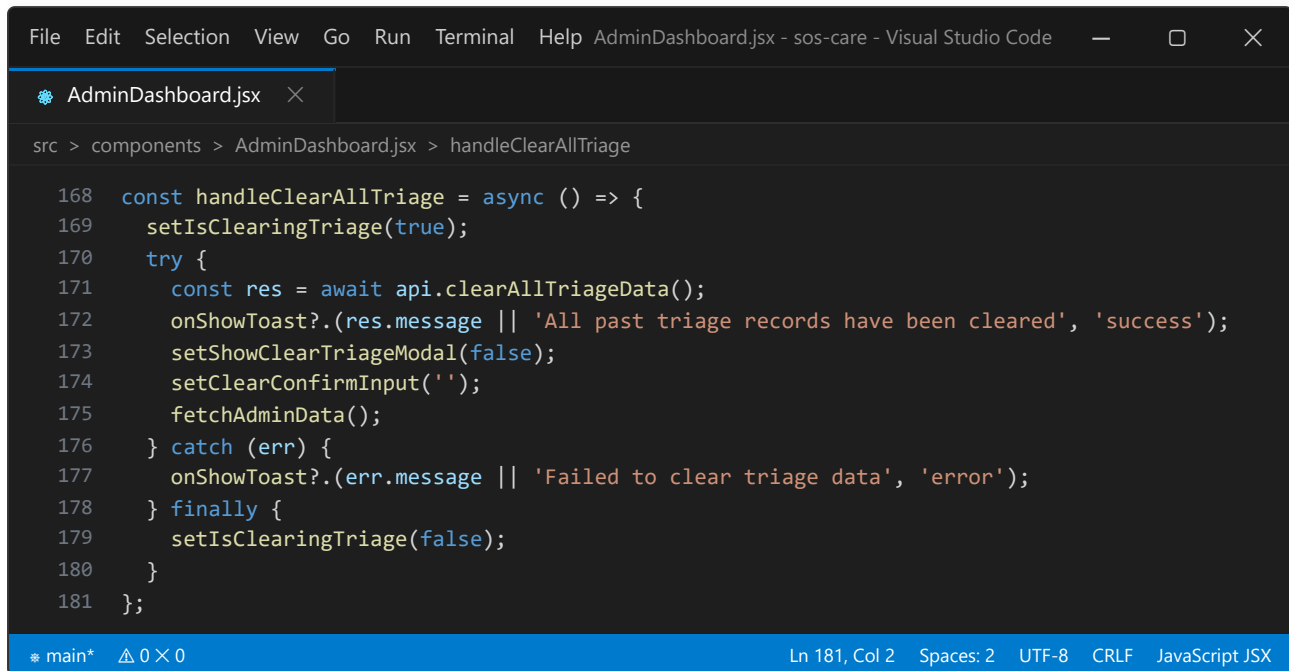
```
File Edit Selection View Go Run Terminal Help AdminDashboard.jsx - sos-care - Visual Studio Code
AdminDashboard.jsx
src > components > AdminDashboard.jsx > handleCreateStaff

131 const handleCreateStaff = async (e) => {
132   e.preventDefault();
133   setIsCreatingStaff(true);
134   try {
135     const created = await api.createStaff(
136       newStaff.name.trim(),
137       newStaff.phone.trim(),
138       newStaff.password,
139       newStaff.staffRole
140     );
141     onShowToast?.(`Staff account created for ${created.name}!`, 'success');
142     setNewStaff({ name: '', phone: '', password: '', staffRole: 'Resident Physician' });
143     fetchAdminData();
144   } finally {
145     setIsCreatingStaff(false);
146   }
147 };

* main* 0x0 Ln 147, Col 2 Spaces: 2 UTF-8 CRLF JavaScript JSX
```

4. Emergency Triage Data Purge Modal (AdminDashboard.jsx):

- Implemented a two-step type-to-confirm modal (CLEAR) allowing administrators to safely wipe test triage cases while safeguarding user and doctor credentials.



The screenshot shows a Visual Studio Code editor window with the file 'AdminDashboard.jsx' open. The breadcrumb navigation indicates the file path is 'src > components > AdminDashboard.jsx > handleClearAllTriage'. The code is a JavaScript function named 'handleClearAllTriage' defined as an async arrow function. It starts by calling 'setIsClearingTriage(true)'. Inside a 'try' block, it calls 'api.clearAllTriageData()', then 'onShowToast?.(res.message || 'All past triage records have been cleared', 'success')', 'setShowClearTriageModal(false)', 'setClearConfirmInput(')', and 'fetchAdminData()'. A 'catch' block handles errors by calling 'onShowToast?.(err.message || 'Failed to clear triage data', 'error')'. A 'finally' block calls 'setIsClearingTriage(false)'. The function ends with a semicolon. The status bar at the bottom shows 'main*' with a diff icon, 'Ln 181, Col 2', 'Spaces: 2', 'UTF-8', 'CRLF', and 'JavaScript JSX'.

```
168 const handleClearAllTriage = async () => {
169   setIsClearingTriage(true);
170   try {
171     const res = await api.clearAllTriageData();
172     onShowToast?.(res.message || 'All past triage records have been cleared', 'success');
173     setShowClearTriageModal(false);
174     setClearConfirmInput('');
175     fetchAdminData();
176   } catch (err) {
177     onShowToast?.(err.message || 'Failed to clear triage data', 'error');
178   } finally {
179     setIsClearingTriage(false);
180   }
181 };
```